

Выполнение запросов в БД

Для отправки синхронных запросов есть два вида API:

1. низкоуровневый - через [IStatement](#);
2. более высокоуровневый - через [SqlQuery](#).

IStatement

IStatement создается с помощью экземпляра класса IDatabase через CreateStatement:

- [C++](#):

```
std::unique_ptr< dbi::IStatement > stmt = dbi::GetConnectedDataDB()->CreateStatement();
```

- [Python](#):

```
stmt = sbis.GetConnectedDatabase().CreateStatement()
```

Для выполнения запросов у IStatement есть две группы методов:

1. SetParam - для задания [параметров](#).

- [C++](#):

```
stmt->SetParam( 1, L"John" );
```

- [Python](#):

```
stmt.SetParam( 1, "John" )
```

Параметры нумеруются с единицы, поддерживаются только позиционные, а не именованные. Параметр хранится в экземпляре IStatement до тех пор, пока его значение не будет "перебито". Сбросить значение всех параметров можно с помощью метода ClearParams().

2. Exec - для исполнения запросов.

- [C++](#):

```
1 std::shared_ptr< ICursor > result = stmt->Exec( L"select * from  
  users where name = $1" );  
2 std::shared_ptr< ICursor > result = stmt->Exec( L"insert into  
  users( name ) values( $1 )" );  
3 std::shared_ptr< ICursor > result = stmt->Exec( L"update users set  
  surname='Doe' where name=$1" );  
4 std::shared_ptr< ICursor > result = stmt->Exec( L"delete from users  
  where name=$1" );
```

- [Python](#):

```
1 result = stmt.Exec( "select * from users where name = $1" )  
2 result = stmt.Exec( "insert into users( name ) values( $1 )" )  
3 result = stmt.Exec( "update users set surname='Doe' where name=$1"  
  )  
4 result = stmt.Exec( "delete from users where name=$1" )
```

Результат метода Exec - объект ICursor или null (определяется характером запроса).

При отправке простого запроса мы получаем ответ `ICursor` с форматом `RecordFormat`, который строится на основе данных из БД и адаптируется под нашу систему типов в терминах платформы.

При таком подходе каких-то продвинутых типов платформы для формата быть не может, например, [Перечисляемое](#). При использовании этого API мы не получим в составе результата перечисляемое, т.к. на уровне БД полей с таким типом данных нет.

Есть другая перегрузка `Exec` - с указанием ожидаемого формата. В этой ситуации передается снаружи формат, который мы ожидаем получить. Тогда система приводит данные, полученные от СУБД к этому формату (если это возможно) или генерирует исключение.

```
std::shared_ptr< ICursor > Exec( const String& stmt, std::shared_ptr< const RecordFormat > format );
```

Также в метод `Exec` можно передать [StopToken](#). При срабатывании токена прерывания запрос отменяется.



Специфика при работе с SQLite

В отличие от PostgreSQL, в SQLite при выполнении модифицирующих запросов (`insert/update/delete`) необходимо перебрать курсор методом `Next`, пока он не вернет `NULL`, иначе запрос фактически не будет выполнен.

SqlQuery

`IStatement` является достаточно низкоуровневым API. Для выполнения даже простого запроса необходимо написать несколько строк кода. Для сокращения записи может применяться семейство методов `SqlQuery`. При применении данного семейства простой запрос может быть записан в одну строку.

Общая схема `SqlQuery`:

```
SqlQuery( запрос, параметры... )
```

В простом случае `SqlQuery` возвращает `RecordSet`, как для C++, так и для Python.

- [C++:](#)

```
RecordSet res = SqlQuery( L"select * from \"Пользователь\" where \"Логин\" = $1", L"Ivan" );
```

- [Python:](#)

```
res = SqlQuery( """select * from \"Пользователь\" where \"Логин\" = $1""", "Ivan" )
```

Дополнительно можно передать:

1. экземпляр `IDatabase` (в качестве первого параметра), к которой хотим сделать запрос.

- C++:

```
RecordSet res = SqlQuery( *dbi::GetConnectedDataDB( L"users" ), L"select * from \"Пользователь\" where \"Логин\" = $1", L"Ivan" );
```

- Python:

```
es = SqlQuery( GetConnectedDatabase( "users" ), """select * from \"Пользователь\" where \"Логин\" = $1""", "Ivan" )
```

2. формат ожидаемых данных.

3. `StopToken`.

Также в C++ доступен [типизированный API для выполнения SQL-запросов](#).